Contrastive Attribution in the Wild: An Interpretability Analysis of LLM Failures on Realistic Benchmarks

Anonymous Authors[1]

Abstract

Interpretability tools are increasingly used to analyze failures of Large Language Models (LLMs), yet prior work largely focuses on short prompts or toy settings, leaving their behavior on commonly used benchmarks underexplored. To address this gap, we study contrastive, LRP-based attribution as a practical tool for analyzing LLM failures in realistic settings. We formulate failure analysis as *contrastive attribution*, attributing the logit difference between an incorrect output token and a correct alternative to input tokens and internal model states, and introduce an efficient extension that enables construction of cross-layer attribution graphs for long-context inputs. Using this framework, we conduct a systematic empirical study across benchmarks, comparing attribution patterns across datasets, model sizes, and training checkpoints. Our results show that this token-level contrastive attribution can yield informative signals in some failure cases, but is not universally applicable, highlighting both its utility and its limitations for realistic LLM failure analysis.

1. Introduction

While the capabilities of Large Language Models (LLMs) have improved substantially in recent years, their deployment in real-world scenarios still reveals a wide range of failures (Chang et al., 2024). Systematically debugging these failures is therefore crucial, both for understanding their underlying causes and for identifying actionable directions for further model improvement.

Existing analyses of LLM failures predominantly rely on *behavioral analysis*, which examines model inputs, outputs, and responses to perturbations. Such analyses are effective at identifying failures, categorizing error types, and measuring sensitivity to prompt variations, but they primarily characterize *what* fails rather than *why* failures occur. A key limitation of this paradigm is its inherent ambiguity: similar behaviors can arise from distinct internal causes, such as underweighting relevant context tokens, overweighting irrelevant tokens, or biases embedded in internal representations. Disambiguating these causes is essential for effective model improvement and motivates interpretability-based analysis (Luo & Specia, 2024), which goes beyond behavioral outcomes by exposing internal preference formation, attributing decisions to input tokens and hidden states, and tracing how influence propagates across model layers.

On the interpretability side, substantial progress has been made for transformer-based models (Ferrando et al., 2024). However, applications to LLMs have largely been limited to toy settings, short inputs, or highly constrained prompt templates (Wang et al., 2023; Meng et al., 2022; Lindsey et al., 2025). Interpretability analyses of realistic failures, such as those arising from standard benchmarks or real-world deployments, remain scarce. This limitation is partly attributable to the open-ended nature of such failures and their long input contexts, which can span thousands of tokens (e.g., in agentic tasks) and exceed the computational limits of many existing interpretability tools.[1] Although recent work has begun to address scalability challenges in mechanistic interpretability (Rosser et al., 2025), direct applications of interpretability methods to analyze realistic LLM failure cases remain limited, leaving their practical utility insufficiently understood.

Motivated by both the need for deeper failure debugging and the desire to establish the practical relevance of model interpretability, we investigate the use of interpretability methods for analyzing failures on *standard benchmarks* (e.g., GAIA2 (Andrews et al., 2025)) as a first step toward more realistic settings. We focus on the central research question: *Can interpretability analysis provide practical value for LLM failure case analysis?* In particular, we study scenarios that are difficult to resolve through behavioral analysis

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

[1]For example, the *TransformerLens* package (Nanda & Bloom, 2022) sets the default context length to 2048 tokens for the Qwen3 model series (Yang et al., 2025).

alone: (1) when a model produces an incorrect token, can interpretability reveal clues about the underlying decision process; (2) when a stronger model outperforms a weaker one, can interpretability expose evidence that the stronger model corrects the weaker model's failure modes; and (3) across training checkpoints, can interpretability reflect the process by which models learn to make fewer mistakes.

To explore these questions, we formulate failure analysis as a problem of explaining why a model prefers an incorrect output token over a more correct alternative. We cast this as **Contrastive Attribution** (Yin & Neubig, 2022), attributing the logit difference between an incorrect token and a correct candidate to input tokens and internal model states. We build on *AttnLRP* (Achtibat et al., 2024), a state-of-the-art variant of the Layer-wise Relevance Propagation (LRP) framework (Bach et al., 2015), and introduce an efficient extension that enables the construction of cross-layer hidden-state attribution graphs for long-context inputs. Using this framework, we conduct a systematic empirical study on tens of failure cases collected over multiple benchmarks, comparing attribution patterns across datasets, model sizes, and training checkpoints. Our results demonstrate that token-level contrastive attribution can yield informative signals for certain classes of failures, but is not universally effective, thereby clarifying both the promise and the limitations of interpretability methods for realistic LLM failure analysis.

Our contributions are summarized as follows:

- We formulate LLM failure analysis as a token-level *contrastive attribution* problem, enabling the application of model interpretability methods to analyze failures in a more realistic setting than prior work.
- We introduce an efficient extension of LRP-based attribution that enables scalable construction of cross-layer hidden-state attribution graphs for long-context inputs.
- We conduct a systematic interpretability-based failure attribution analysis across common benchmarks, characterizing attribution patterns across datasets, model sizes, and training checkpoints in realistic failure scenarios.[2]

2. Related Work

Analyzing LLM Failures. LLM failures are often encountered and studied after benchmark evaluation. Such analyses have revealed systematic weaknesses in areas including natural language inference, semantic and abstract reasoning, robustness, and bias (Chang et al., 2024). However, these studies largely treat LLMs as black boxes, relying on output-level metrics without examining the internal mechanisms that lead to failures.

Another prominent line of research studies LLM failures under the umbrella of *hallucination*. Empirical work in this area characterizes hallucination behaviors (Maynez et al., 2020) as covered by recent surveys (Ji et al., 2023; Huang et al., 2025; Zhang et al., 2025c). To facilitate controlled analysis, several hallucination-focused benchmarks have been proposed (Lin et al., 2022; Li et al., 2023; Bang et al., 2025). While these datasets are valuable for isolating hallucination phenomena, the resulting failures are tightly coupled to dataset design and differ from errors that arise organically in standard benchmark evaluations.

Interpretability methods have also been applied to study hallucination-related behaviors, but predominantly in synthetic or simplified settings. Prior work (Meng et al., 2022; Dai et al., 2022; Geva et al., 2021; Meng et al., 2023; Yao et al., 2024; Lindsey et al., 2025) often relies on short prompts or highly structured formats, such as subject–relation–object templates (Meng et al., 2022). These settings abstract away much of the complexity present in realistic, open-ended benchmark failures.

Research on LLM reasoning failures has also gained renewed attention with the emergence of large reasoning models (Li et al., 2025). Recent studies categorize reasoning errors (Song et al., 2025), monitor reasoning processes (Baker et al., 2025), steer or interpret reasoning via sparse features (Galichin et al., 2025), and debug reasoning failures (Zhang et al., 2025a). However, most of these works focus primarily on reasoning tokens, rather than connecting failures to broader benchmark-defined errors.

Lastly, a growing body of research work examines failures in LLM-powered agent systems, including failure taxonomies (Cemri et al., 2025), failure localization (Zhang et al., 2025b), and agent debugging methods (Ma et al., 2025). These studies primarily operate at the system or agent level and do not directly investigate the underlying causes of failure by incorporating the base LLMs.

Input Attribution. This work adopts LRP to attribute the final token logit difference to input tokens and internal states. More broadly, input attribution methods aim to localize model behavior by estimating the contribution of individual input tokens to model predictions (Ferrando et al., 2024). A prominent class of approaches is gradient-based attribution, including gradient norms (Simonyan et al., 2014), $gradient \times input$ (Denil et al., 2015), integrated gradients (Sundararajan et al., 2017), and SmoothGrad (Smilkov et al., 2017). Another widely studied family comprises perturbation-based methods, which estimate input importance by adding noise or ablating input elements and measuring the resulting change in model predictions (Lundberg

[2]Our code and data are available at: https://anonymous.4open.science/r/contrastive_attribution_in_the_wild-7201/.

& Lee, 2017; Fong & Vedaldi, 2017; Covert et al., 2021).

Beyond these, attention-based and context-mixing methods aim to decompose the final-layer token representations used for prediction into layer-wise contributions by explicitly tracking how attention mixes information across tokens (Kobayashi et al., 2021; Ferrando et al., 2023). Within the LRP framework, several variants beyond AttnLRP have also been proposed (Ali et al., 2022). Among existing approaches, it was shown that AttnLRP achieved state-of-the-art faithfulness in input token attribution for transformer models (Achtibat et al., 2024). We therefore adopt AttnLRP as the interpretability method in this work.

Recent studies have also focused on improving the computational efficiency of attribution methods, which often incur substantial overhead (Liu et al., 2025; Arras et al., 2025). Our efficient extension of AttnLRP builds on the backpropagation-based method in (Arras et al., 2025).

Causal Patching, Circuit Discovery, and Attribution Graph. Beyond input-level attribution, previous work has also investigated attribution to internal model states and the identification of critical computation paths (*circuits*) that give rise to a target output. A prominent class of methods relies on causal patching, including activation patching (Heimersheim & Nanda, 2024; Zhang & Nanda, 2024), attribution patching (Nanda, 2023; Kramár et al., 2024), and path patching (Wang et al., 2023; Goldowsky-Dill et al., 2023). These approaches introduce paired *clean* and *corrupted* prompts and identify critical model components by selectively patching activations from one run into the other. While such contrastive setups provide strong causal control, constructing suitable clean–corrupted prompt pairs is often nontrivial, especially when analyzing real-world failures.

Another line of work trains sparse autoencoders (e.g., transcoders) to obtain locally linearized representations of model activations, enabling circuit discovery, attribution graph construction, and mechanistic analysis (Dunefsky et al., 2024; Ameisen et al., 2025; Zhao et al., 2025; Dai et al., 2025). Although these methods can reveal fine-grained, neuron-level structure, they are computationally intensive, typically restricted to short prompts, and require substantial human expertise for effective analysis. Moreover, the need to train a separate transcoder for each model limits their scalability across different LLMs.

Attribution graphs can also be constructed using decomposition-based methods such as LRP and attention-based approaches (Ferrando & Voita, 2024; Rosser et al., 2025). These techniques are generally more computationally efficient while maintaining reasonable faithfulness in their attributions. Given the long-context requirements inherent in realistic failure analysis, we therefore develop attribution graphs based on AttnLRP in this work.

3. Methodology

3.1. Contrastive Attribution via LRP

Model. We consider transformer-based autoregressive LLMs (Vaswani et al., 2017). Let $\mathcal{V}$ denote the vocabulary with $|\mathcal{V}|$ tokens and let d be the hidden dimension. The model comprises an embedding layer with matrix $\mathbf{W}_E \in \mathbb{R}^{|\mathcal{V}|\times d}$, followed by L transformer blocks indexed by l. An unembedding matrix $\mathbf{W}_U \in \mathbb{R}^{d\times|\mathcal{V}|}$ maps final hidden representations to vocabulary logits.

Given an input token sequence $\mathbf{t}_{\leq i} = (t_0, \ldots, t_i)$ with $t_j \in \mathcal{V}$, the embedding layer produces token embeddings $\mathbf{h}_{-1} = \mathbf{W}_E[\mathbf{t}_{\leq i}]$. Hidden states are propagated through the transformer stack as

$$\mathbf{h}_l = \mathrm{Transformer}_l(\mathbf{h}_{l-1}), \quad l = 0, \ldots, L-1. \tag{1}$$

Let $\mathbf{h}_{L-1}^{(i)} \in \mathbb{R}^d$ denote the final-layer hidden state at position i. The logits for next-token prediction are

$$\boldsymbol{\ell}_{i+1} = \mathbf{h}_{L-1}^{(i)\top}\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}|}. \tag{2}$$

Contrastive attribution objective. We formulate failure analysis as a token-level *contrastive attribution* problem. Given an incorrect target token t_{tgt} and a contrast alternative t_{con}, we attribute the logit difference between them to input token embeddings and hidden states across layers. The contrastive logit difference is

$$\begin{aligned}\Delta\ell &:= \ell_{i+1}(t_{\mathrm{tgt}}) - \ell_{i+1}(t_{\mathrm{con}}) \\ &= \mathbf{h}_{L-1}^{(i)\top}\big(\mathbf{W}_U[:, t_{\mathrm{tgt}}] - \mathbf{W}_U[:, t_{\mathrm{con}}]\big),\end{aligned} \tag{3}$$

where $\mathbf{W}_U[:, t]$ denotes the unembedding vector for token t. This contrastive formulation closely reflects practical debugging scenarios (asking why the model prefers a specific incorrect token over a plausible alternative) and removes shared, failure-irrelevant computation, yielding more salient and interpretable explanations (Yin & Neubig, 2022).

Layer-wise Relevance Propagation. To attribute $\Delta\ell$ to intermediate representations, we employ Layer-wise Relevance Propagation (LRP) (Bach et al., 2015). LRP is an additive explanation method that decomposes a scalar quantity R_j into contributions from input features $\mathbf{x} = \{x_i\}_{i=1}^N$: $R_j = \sum_{i=1}^N R_{i\leftarrow j}$, where $R_{i\leftarrow j}$ denotes the relevance assigned to feature x_i for explaining R_j. When a feature i contributes to multiple downstream quantities j, its total relevance is obtained by aggregation $R_i = \sum_j R_{i\leftarrow j}$.

Applying LRP to our setting, we treat the contrastive logit difference $\Delta\ell$ as the attribution target and propagate relevance backward through all transformer layers to the embedding layer. This yields a relevance vector $\mathbf{R}_i^{(l)} \in \mathbb{R}^d$

associated with each hidden state $\mathbf{h}_l^{(i)}$:

$$\mathbf{R}_i^{(l)} \leftrightarrow \mathbf{h}_l^{(i)}, \tag{4}$$

$$\mathbf{R}_i^{(l)} = \sum_j \mathbf{R}_{\mathbf{h}_l^{(i)} \leftarrow \mathbf{h}_{l+1}^{(j)}}, \tag{5}$$

where $\mathbf{R}_{\mathbf{h}_l^{(i)} \leftarrow \mathbf{h}_{l+1}^{(j)}}$ denotes the relevance propagated from hidden state $\mathbf{h}_{l+1}^{(j)}$ in the subsequent layer. Collectively, these relevances define an **Attribution Graph**, whose nodes are $\mathbf{R}_i^{(l)}$ and whose edges correspond to propagated relevances.

LRP variant. Because transformers are highly non-linear, LRP relies on module-specific propagation rules that locally linearize model components. Different rules yield different LRP variants. As discussed earlier, although AttnLRP relaxes strict layer-wise relevance conservation, we adopt *AttnLRP* (Achtibat et al., 2024) due to its superior attribution faithfulness compared to alternative attribution methods.

3.2. Efficient Attribution Graph Construction

Recent work shows that the relevance of input embeddings and hidden states under LRP can be efficiently computed via a modified *gradient*$\times$*input* formulation, enabling a single backward pass using standard automatic differentiation frameworks (Achtibat et al., 2024; Arras et al., 2025). While this approach yields relevance scores for hidden states across layers, it does not directly provide the *propagation structure* required to construct an attribution graph, i.e., how relevance flows between hidden states across layers.

Naively constructing the attribution graph is computationally prohibitive. To recover relevance propagation details, one would need to treat each hidden-state relevance in layer $l+1$ as a separate attribution target and backpropagate it to layer l. Given the large cardinality of $\{\mathbf{R}_i^{(l)}\}$, this results in an excessive number of backward passes.

To address this challenge, we leverage a batching trick that reuses the batch dimension to pack multiple attribution targets into a single backward pass, following recent work on attribution graph construction with sparse features in transcoders (Hanna et al., 2025). This approach exploits GPU vectorization to efficiently recover relevance propagation between layers. Details are provided in Appendix A. To further improve efficiency, we prune attribution targets with relevance below a fixed threshold and remove edges with negligible relevance during graph construction. The graph pruning strategy is described in Appendix B.

3.3. Coarse-to-Fine Attribution Analysis

The above formulation yields relevance at the level of individual hidden-state components, enabling neuron-level analysis (Achtibat et al., 2024). In practice, we also often aggregate relevance at the hidden-state level by summing over hidden state dimensions, $R_i^{(l)} = \sum_{k=1}^{d} \mathbf{R}_{i,k}^{(l)}$, where $\mathbf{R}_{i,k}^{(l)}$ denotes the k-th component of $\mathbf{R}_i^{(l)}$. We use this aggregated relevance to analyze input-token attribution patterns, e.g., via heatmaps (as shown in Figure 2).

A coarse-grained treatment is also essential for attribution graph analysis. Constructing attribution graphs is computationally expensive, particularly for long prompts. To scale LRP-based interpretability to large collections of failure cases, we first construct attribution graphs at the hidden-state level by propagating aggregated relevance. After identifying important subgraphs, we optionally refine them by propagating fine-grained, neuron-level relevance within the selected subgraph. This coarse-to-fine strategy differs from prior attribution graph approaches that directly operate at the neuron level (e.g., (Ameisen et al., 2025)). For long-context failure cases, a one-shot neuron-level construction is often computationally infeasible and hinders interpretability.

As an initial step, in this work we focus on the coarse-grained state-level analysis, and leave systematic neuron-level attribution graph analysis to future work.

4. Experiments

4.1. Experimental Setup

Benchmarks. To ensure our benchmark choices align with widely used ones while remaining relevant amid rapidly improving model capabilities, we follow the latest recommendations from HuggingFace (Fourrier et al., 2025). Specifically, we adopt GAIA2 (Andrews et al., 2025) to represent long-horizon agentic tasks, IFEval (Zhou et al., 2023) to evaluate instruction-following, MATH (Hendrycks et al., 2021) for math reasoning, and HumanEval (Chen et al., 2021) for code generation.[3] Together, these benchmarks span diverse domains, context lengths (e.g., 10k+ tokens for GAIA2 traces), and task types (agentic vs. non-agentic).

Models. We primarily use the open-source Qwen3 model series (Yang et al., 2025) for attribution analyses, and the Olmo-3-7B-Think model series (Olmo et al., 2025) to study the evolution of attribution patterns across training checkpoints. Considering our computation resource constraints, we restrict the model sizes to below 8B. Benchmark-specific model configurations are summarized in Table 1.

Failure Case Collection. To generate failure cases for analysis, we employed different model configuration across benchmarks, as summarized in Table 1. Greedy decoding is employed for reproducibility. To ensure balanced analysis

[3] We do not include additional agent-based coding benchmarks, as GAIA2 already captures agentic behavior; instead, we select HumanEval to emphasize code-centric evaluation.

Table 1. Summary of benchmarks and models, number of original failure cases, proportion of clean cases used for attribution analysis, and average input token count. Models highlighted in **bold** are used for failure trace generation.

Benchmark	Target Capability	Model	Failure Cases (Clean Case Rate)	Avg. Token Count
IFEval	Instruction following	Qwen3-**0.6B**/1.7B/4B	156 (13.5%)	54
GAIA2	Agentic; long-context	Qwen3-**4B**	100 (15.0%)	12374
MATH	Math	Qwen3-**0.6B**	45 (35.6%)	116
HumanEval	Coding	Qwen3-**0.6B**	96 (19.79%)	169

across datasets, we evaluated the full HumanEval set due to its small size, while randomly sampling a subset from larger datasets IFEval, GAIA2, and MATH. Moreover, for IFEval, MATH, and HumanEval, we disabled the models' thinking mode to reduce the confounding effects of self-reflection and dynamic reasoning adjustments, which obscure the causal trace of errors and complicate attribution. Prior work has noted similar challenges in analyzing non-monotonic reasoning paths and residual influence from negated thoughts (Xu et al., 2025; Ashury-Tahan et al., 2026).

In contrast, for the GAIA2 benchmark, we retained the default setting with thinking mode enabled, using the official Meta-ARE system (Andrews et al., 2025). This choice was made because GAIA2 involves complex multi-turn tasks where thinking mode aids long-horizon coherence and stable reasoning paths reduce self-overwriting. The number of collected failure cases per benchmark is shown in Table 1.

Contrast Token Pair Identification. After collecting failure cases, we apply a post-processing step to identify contrast token pairs for attribution analysis. This step can be challenging for several reasons: (i) there may be multiple plausible candidates, or none at all, for the target error token; and (ii) multiple choices may exist for the corresponding contrast token. In our implementation, we define the target token as the *first* identifiable token whose generation causes the model output to deviate from the correct trajectory. The contrast token is primarily selected as the highest-ranked alternative during top-k rollout, using both the models that generated the failure traces and stronger auxiliary models to assist identification. Additional details on contrast token pair identification are provided in Appendix C.

Following this procedure, we retain 13.5% of failure cases for IFEval, 15.0% for GAIA2, 35.6% for MATH, and 19.79% for HumanEval. The remaining cases are excluded due to factors such as the inability to identify a valid target token (e.g., unbounded generation), the absence of a suitable contrast token (e.g., no token capable of reversing the incorrect reasoning trajectory), or violation of our logit-difference threshold $\Delta\ell > 1$, which is imposed to avoid weak token preferences and numerical instability. While this filtering yields a set of clean and well-defined failure cases for analysis, it also highlights a limitation of the proposed contrastive attribution method, namely reduced coverage when applied to real-world failure debugging scenarios.

Attribution Analysis. We conduct attribution analysis in two stages. First, we apply input attribution using AttnLRP, producing heatmaps that indicate each input token's positive or negative contribution to the logit difference, as illustrated in Figure 2. Tokens with positive relevance (red) support the target token, whereas tokens with negative relevance (blue) disfavor the target token and instead support the contrast token. Because the beginning-of-sequence (BOS) and other special tokens often receive disproportionately large relevance due to the attention sink effect (Xiao et al., 2024; Cancedda, 2024), we exclude these special tokens (shown in gray in Figure 2) when normalizing relevance scores. Specifically, we normalize by the absolute value of the maximum relevance score among non-special tokens.

Second, we construct attribution graphs using our proposed batch-packed multi-target backpropagation method. During graph construction, we discard nodes with absolute relevance below 0.01 and prune edges using a threshold of 0.85 under the per-layer cumulative mass pruning mode.

With the input attribution heatmaps and the attribution graphs, we perform a detailed manual analysis of identified clean failure cases. The findings are independently verified by two authors. We leave the development of a fully automated attribution analysis pipeline to future work.

4.2. Attribution Analysis of Incorrect Token Preference

We first present attribution analysis results across four benchmarks, using the same model employed for failure trace generation. Our goal is to assess the extent to which attribution analysis can reveal explanatory signals for why the target token is preferred over its contrast token.

To this end, we categorize attribution outcomes as follows:

- **Manifested by Input Attribution (M-IA):** The failure is clearly observable in the input attribution heatmap.
- **No Clue from Input Attribution (NC-IA) + Manifested by Attribution Graph (M-AG):** The failure is not evident from input attribution alone but becomes apparent through attribution graph analysis.
- **No Clue from Both Input Attribution and Attribution Graph (NC-IA+AG):** The failure is not explained by either analysis, demanding finer-grain analysis.

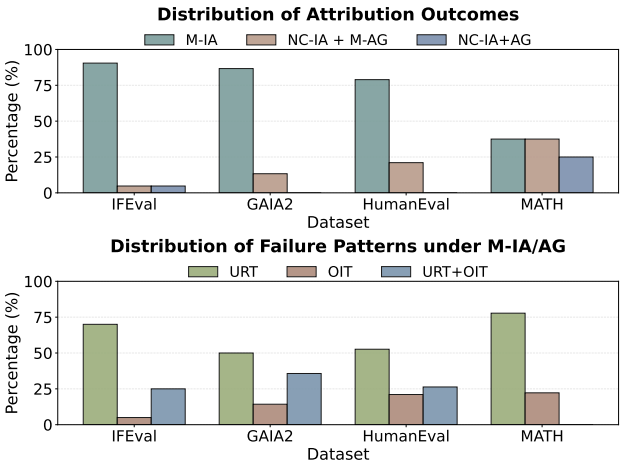

Figure 1. Distribution of attribution outcomes (top) and failure patterns (bottom) across benchmarks.

The resulting categorization for the four benchmarks is shown in the top panel of Figure 1. IFEval, GAIA2, and HumanEval exhibit similar patterns: in the majority of cases, input attribution alone suffices to reveal failure signatures. In contrast, for MATH, a substantial fraction of failures remain unexplained even after attribution graph analysis. This limitation arises because our current attribution graphs operate at the level of aggregated hidden states, whereas numerical reasoning errors often require finer-grained, neuron-level attribution analysis (Lindsey et al., 2025). For example, for the case *"Algebra_9"* in MATH, Qwen3-0.6B incorrectly prefers the digit "0" over "1" following the prior context of "$(1.0175)^{20} \approx 1.4$". Explaining this error may require neuron-level tracing of the underlying power computation.

For cases that are explainable by attribution analysis (i.e., those categorized as M-IA and NC-IA + M-AG), we further identify the following failure patterns:

- **Underweight Relevant Tokens (URT):** The model assigns insufficient relevance to critical input tokens. Figure 2(a) shows an example from IFEval, where Qwen3-0.6B fails to assign negative relevance to the keyword "comma", which is essential for producing the correct output. In contrast, Qwen3-4B assigns clear negative relevance to this token, aligning with correct behavior.
- **Overweight Irrelevant Tokens (OIT):** The model overemphasizes misleading or semantically uninformative tokens. Note that URT does not imply OIT, as AttnLRP does not enforce relevance conservation. As illustrated in Figure 2(b), a HumanEval example shows Qwen3-0.6B assigning high relevance to an irrelevant token "(a", while much less contribution from that token in Qwen3-8B.
- **URT + OIT:** Failures characterized by both underweighting relevant tokens and overweighting irrelevant ones.

Figure 2. Examples of input attribution heatmaps. (a) URT: Qwen3-0.6B underweights the instruction token "commas". (b) OIT: Qwen3-0.6B overweights an irrelevant token "(a". (c) NC-IA: input attribution alone offers limited explanation, motivating attribution graph analysis. Color intensity shows each token's positive (red) or negative (blue) influence on the target–contrast preference.

A fine-grained breakdown of the above failure patterns is shown in the bottom panel of Figure 1. Across all four benchmarks, underweighting relevant tokens emerges as the dominant failure mode. Overweighting irrelevant tokens also contributes substantially, particularly for HumanEval and GAIA2, and is often accompanied by underweighting of relevant tokens (except in MATH).

Lastly, we present a representative failure case in which input attribution offers limited insight, whereas the attribution graph reveals informative internal dynamics. As shown in Figure 2(c), the output already satisfies the instruction to repeat the request and should therefore begin with the token "<<". Instead, the model erroneously outputs the token "First". Although the key tokens "<<" and "brackets" exhibit substantial negative relevance relative to the large positive relevance of "First", the model still yields a large final logit difference of 5.25.

To further analyze this behavior, we examine the attribution graph in Figure 3, which highlights key tokens, layers, and relevance propagation paths. As relevance propagates to higher layers, the competing tokens "First" and "<<" initially show similar growth rates. At Layer 16, the absolute relevance of "<<" increases sharply (from 6.72 to 14.0), even surpassing that of "First", due to aggregated relevance from earlier tokens such as "brackets such as". Beyond Layer 16, "First" contributes several substantial increments to the residual stream at the final token position (Layers 18, 20, and 23), whereas "<<" makes two large contributions (Layers 22 and 24). The cumulative magnitude of relevance

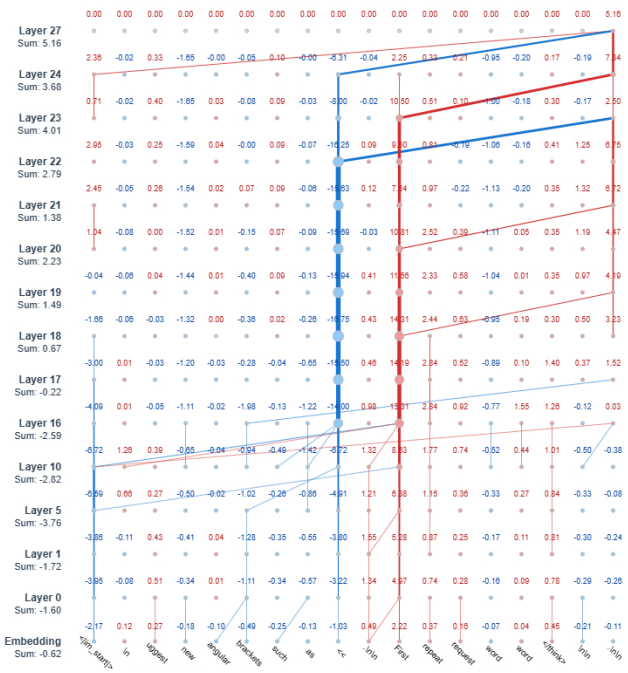

Figure 3. Sample ablated attribution graph for an NC-IA + M-AG case; the expanded attribution graph is provided in Figure 8.

transferred by "First" ultimately exceeds that of "<<", resulting in a positive final logit difference and the observed erroneous output. Overall, this case illustrates how attribution graphs expose layer-wise relevance interactions that are invisible to input-level attribution, providing a more fine grained explanation of the model's final decision. More example attribution graphs are given in Appendix D.

4.3. Attribution Shifts with Model Scaling

We next investigate whether scaling model size can resolve the failures observed in smaller models and, if so, whether such improvements are supported by interpretability evidence. Aligning performance gains with interpretable mechanisms is crucial to verify that scaling improves models in the intended direction rather than exploiting spurious cues.

To this end, we apply larger Qwen3 models (1.7B and 4B) to the failure cases on IFEval identified for Qwen3-0.6B in the previous subsection. As shown in Figure 4, larger models consistently yield more negative logit differences for the contrast token pairs. In the plots comparing "0.6B vs. 1.7B" and "0.6B vs. 4B", most data points lie below the $y = x$ line, indicating that larger models tend to correct incorrect token preferences made by the smaller model. A similar trend is observed when comparing 1.7B and 4B, where many points exhibit further negative shifts (green downward arrows). While the majority of failures are corrected, as reflected by the concentration of points in the green shaded region, a

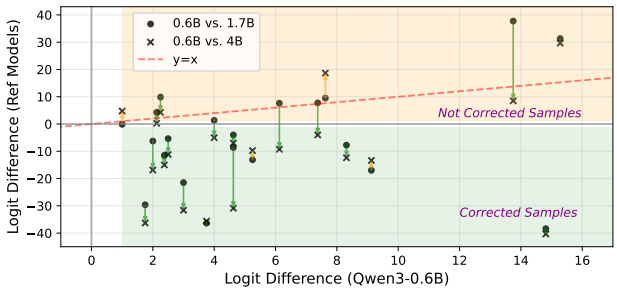

Figure 4. Logit difference comparisons between Qwen3-0.6B and larger models (1.7B and 4B) on IFEval failure cases. Most points lie below the $y = x$ line, indicating that larger models tend to correct failures in the smaller model. Green regions denote corrected samples, while yellow regions indicate remaining failures.

subset of cases remains uncorrected (yellow region).

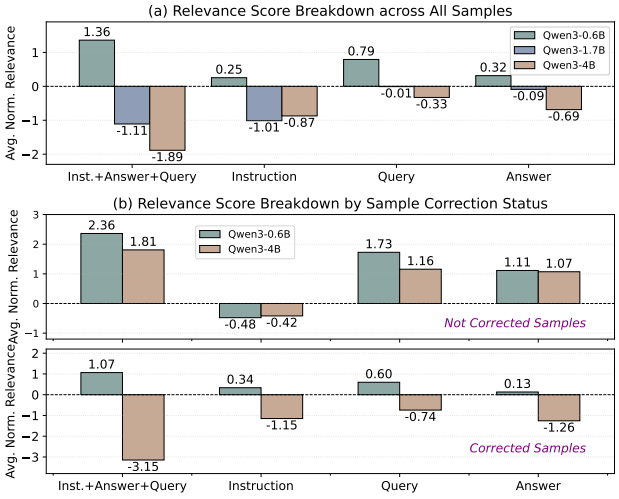

Figure 5. Input attribution relevance score breakdown across prompt segments. **(a)** Average normalized relevance over all samples. **(b)** Relevance breakdown by correction status.

We next examine whether these improvements are supported by interpretability analyses. Specifically, we perform input attribution for all three models on the same cases and compute normalized relevance score distributions over three prompt segments: *Instruction* (constraint-related tokens, e.g., "not allowed to use any commas"), *Query* (the task description), and *Answer* (tokens after `</think>`). The results are shown in Figure 5. The top panel demonstrates that total relevance scores become increasingly negative as model size increases from 0.6B to 1.7B and 4B. Notably, the Instruction segment, which exhibits non-negative relevance in the 0.6B model, becomes clearly negative in the larger models. In addition, both Query and Answer segments also show more negative relevance, suggesting that larger models are more attentive to task requirements and less driven by superficial token continuation from prior answer tokens.

This pattern becomes more pronounced when separating corrected and uncorrected samples, as shown in the bottom panel of Figure 5. For corrected samples, all three segments exhibit negative relevance scores, whereas uncorrected samples display nearly identical relevance distributions across model sizes. These findings indicate that failure correction in larger models is associated with systematic and interpretable shifts in attribution patterns. Overall, the results provide evidence that scaling model size leads to genuine improvements and strengthens confidence in model scaling as a principled approach to enhancing performance.

4.4. Evolution of Failure Attribution Across Training

Beyond studying the model scaling effects, we further apply them to investigate how model behavior evolves over the course of training. Specifically, we analyze input attribution patterns across post-training checkpoints of the Olmo-3-7B-Think model series (Olmo et al., 2025).

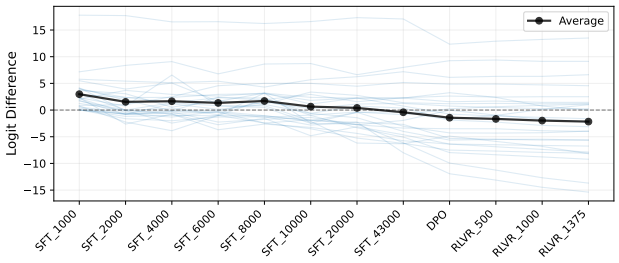

Figure 6. Evolution of logit differences across training checkpoints for Olmo-3-7B-Think on IFEval failure cases identified at `SFT_1000`. Each line corresponds to an individual sample, with the bold line denoting the average.

We begin by identifying failure samples on IFEval using an early checkpoint, `SFT_1000`, at step 1000 of the supervised fine-tuning (SFT) stage. After extracting the corresponding contrast token pairs, we perform input attribution for these same cases across later checkpoints, including eight checkpoints during SFT, one checkpoint after Direct Preference Optimization (DPO), and three checkpoints in the final Reinforcement Learning with Verifiable Rewards (RLVR) stage. As shown in Figure 6, we observe a steady reduction in overall logit differences as training progresses.

Analogous to our analysis of model scaling, we further examine how attribution patterns evolve across checkpoints by computing relevance score breakdowns over prompt segments, as shown in Figure 7. Two key observations emerge. First, the most substantial relevance changes occur during the early SFT phase, particularly between `SFT_1000` and `SFT_2000`. This reduction is primarily driven by decreased relevance in the *Query* and *Answer* segments, suggesting that early SFT helps the model become more attentive to task semantics while reducing reliance on superficial token continuation from prior answers. A second notable relevance drop appears during the middle SFT stage, between `SFT_8000` and `SFT_10000`, again largely attributable to changes in the Query and Answer segments. Importantly, these inflection points align closely with the corresponding reductions in logit differences observed in Figure 6.

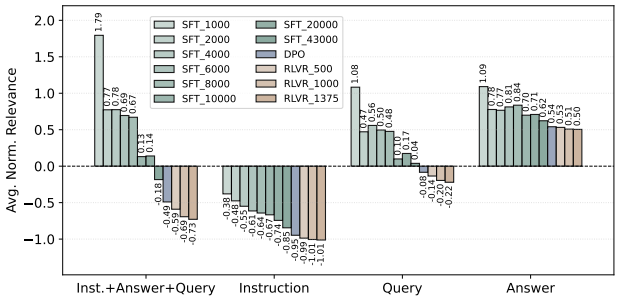

Figure 7. Input attribution relevance score breakdown across training checkpoints by prompt segment. Relevance is shown for *Instruction*, *Query*, and *Answer* segments.

Second, while relevance in Query and Answer exhibits pronounced shifts, relevance associated with *Instruction* decreases more steadily across checkpoints. Notably, comparing the final SFT checkpoint (`SFT_43000`) with the post-DPO checkpoint reveals that DPO further pushes relevances in all three segments toward more negative values.

Overall, these interpretability-based analyses provide fine-grained evidence that training progressively corrects failures through systematic attribution shifts. This highlights the utility of attribution-based diagnostics as a practical tool to monitor and debug model behavior throughout training.

5. Conclusion

We studied contrastive, LRP-based attribution as a practical tool for analyzing LLM failures on realistic benchmarks. By framing failure analysis as explaining why a model prefers an incorrect token over a correct alternative, and introducing an efficient method for constructing cross-layer attribution graphs, we applied interpretability at scales and settings largely unexplored by prior work. Across benchmarks, model sizes, and training checkpoints, we find that contrastive attribution can reveal meaningful failure patterns in many cases (e.g., underweighting relevant context) and that improvements from model scaling and continued training are often accompanied by systematic, interpretable shifts in attribution. Meanwhile, a non-trivial fraction of failures remain unexplained, particularly in math reasoning, highlighting limitations of our coarse grain attribution methods. Overall, our results suggest that interpretability can offer concrete diagnostic value for realistic LLM failure analysis, while also underscoring the need for more expressive and scalable tools to fully explain complex model errors.

Impact Statement

This paper presents work aimed at advancing the understanding and practical use of interpretability methods for analyzing failures in large language models. By studying how attribution techniques behave on realistic benchmarks, our work may contribute to improved debugging, evaluation, and transparency of deployed models. While such tools could support more reliable and accountable AI systems, they may also be misinterpreted or overtrusted if used without awareness of their limitations. We do not identify any immediate negative societal impacts beyond those already associated with LLMs and interpretability research, and we believe the broader implications of this work are consistent with established directions in machine learning research.

References

Achtibat, R., Hatefi, S. M. V., Dreyer, M., Jain, A., Wiegand, T., Lapuschkin, S., and Samek, W. Attnlrp: Attention-aware layer-wise relevance propagation for transformers. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=emtXYlBrNF.

Ali, A., Schnake, T., Eberle, O., Montavon, G., Müller, K., and Wolf, L. XAI for transformers: Better explanations through conservative propagation. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 435–451. PMLR, 2022. URL https://proceedings.mlr.press/v162/ali22a.html.

Ameisen, E., Lindsey, J., Pearce, A., Gurnee, W., Turner, N. L., Chen, B., Citro, C., Abrahams, D., Carter, S., Hosmer, B., Marcus, J., Sklar, M., Templeton, A., Bricken, T., McDougall, C., Cunningham, H., Henighan, T., Jermyn, A., Jones, A., Persic, A., Qi, Z., Ben Thompson, T., Zimmerman, S., Rivoire, K., Conerly, T., Olah, C., and Batson, J. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/methods.html.

Andrews, P., Benhalloum, A., Bertran, G. M.-T., Bettini, M., Budhiraja, A., Cabral, R. S., Do, V., Froger, R., Garreau, E., Gaya, J.-B., Laurençon, H., Lecanu, M., Malkan, K., Mekala, D., Ménard, P., Mialon, G., Piterbarg, U., Plekhanov, M., Rita, M., Rusakov, A., Scialom, T., Vorotilov, V., Wang, M., and Yu, I. Are: Scaling up agent environments and evaluations, 2025. URL https://arxiv.org/abs/2509.17158.

Arras, L., Puri, B., Kahardipraja, P., Lapuschkin, S., and Samek, W. A close look at decomposition-based xai-methods for transformer language models, 2025. URL https://arxiv.org/abs/2502.15886.

Ashury-Tahan, S., Mai, Y., Bandel, E., Shmueli-Scheuer, M., and Choshen, L. Errormap and erroratlas: Charting the failure landscape of large language models, 2026. URL https://arxiv.org/abs/2601.15812.

Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K.-R., and Samek, W. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PloS one*, 10(7):e0130140, 2015.

Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M. Y., Madry, A., Zaremba, W., Pachocki, J., and Farhi, D. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. URL https://arxiv.org/abs/2503.11926.

Bang, Y., Ji, Z., Schelten, A., Hartshorn, A., Fowler, T., Zhang, C., Cancedda, N., and Fung, P. Hallulens: Llm hallucination benchmark, 2025. URL https://arxiv.org/abs/2504.17550.

Cancedda, N. Spectral filters, dark signals, and attention sinks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4792–4808, 2024.

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why do multi-agent llm systems fail?, 2025. URL https://arxiv.org/abs/2503.13657.

Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., and Xie, X. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.*, 15(3), March 2024. ISSN 2157-6904. doi: 10.1145/3641289. URL https://doi.org/10.1145/3641289.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M.,

Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Covert, I., Lundberg, S. M., and Lee, S. Explaining by removing: A unified framework for model explanation. *J. Mach. Learn. Res.*, 22:209:1–209:90, 2021. URL https://jmlr.org/papers/v22/20-1316.html.

Dai, D., Dong, L., Hao, Y., Sui, Z., Chang, B., and Wei, F. Knowledge neurons in pretrained transformers. In Muresan, S., Nakov, P., and Villavicencio, A. (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8493–8502, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.581. URL https://aclanthology.org/2022.acl-long.581/.

Dai, X., Guo, K., Lo, C.-H., Zeng, S., Ding, J., Luo, D., Mukherjee, S., and Tang, J. Graphghost: Tracing structures behind large language models, 2025. URL https://arxiv.org/abs/2510.08613.

DeepSeek-AI, Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., et al. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556.

Denil, M., Demiraj, A., and de Freitas, N. Extraction of salient sentences from labelled documents, 2015. URL https://arxiv.org/abs/1412.6815.

Dunefsky, J., Chlenski, P., and Nanda, N. Transcoders find interpretable LLM feature circuits. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

Ferrando, J. and Voita, E. Information flow routes: Automatically interpreting language models at scale. In Al-Onaizan, Y., Bansal, M., and Chen, Y. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pp. 17432–17445. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.EMNLP-MAIN.965. URL https://doi.org/10.18653/v1/2024.emnlp-main.965.

Ferrando, J., Gállego, G. I., Tsiamas, I., and Costa-jussà, M. R. Explaining how transformers use context to build predictions. In Rogers, A., Boyd-Graber, J. L., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pp. 5486–5513. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.ACL-LONG.301. URL https://doi.org/10.18653/v1/2023.acl-long.301.

Ferrando, J., Sarti, G., Bisazza, A., and Costa-jussà, M. R. A primer on the inner workings of transformer-based language models, 2024. URL https://arxiv.org/abs/2405.00208.

Fong, R. C. and Vedaldi, A. Interpretable explanations of black boxes by meaningful perturbation. In *IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017*, pp. 3449–3457. IEEE Computer Society, 2017. doi: 10.1109/ICCV.2017.371. URL https://doi.org/10.1109/ICCV.2017.371.

Fourrier, C., Frere, T., Penedo, G., and Wolf, T. The llm evaluation guidebook, 2025. URL https://huggingface.co/spaces/OpenEvals/evaluation-guidebook#recommendations.

Galichin, A., Dontsov, A., Druzhinina, P., Razzhigaev, A., Rogov, O. Y., Tutubalina, E., and Oseledets, I. I have covered all the bases here: Interpreting reasoning features in large language models via sparse autoencoders, 2025. URL https://arxiv.org/abs/2503.18878.

Geva, M., Schuster, R., Berant, J., and Levy, O. Transformer feed-forward layers are key-value memories. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t. (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5484–5495, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.446. URL https://aclanthology.org/2021.emnlp-main.446/.

Goldowsky-Dill, N., MacLeod, C., Sato, L., and Arora, A. Localizing model behavior with path patching, 2023. URL https://arxiv.org/abs/2304.05969.

Hanna, M., Piotrowski, M., Lindsey, J., and Ameisen, E. Circuit-tracer: A new library for finding feature circuits. In Belinkov, Y., Mueller, A., Kim, N., Mohebbi, H., Chen, H., Arad, D., and Sarti, G. (eds.), *Proceedings of the 8th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 239–249, Suzhou, China,

November 2025. Association for Computational Linguistics. ISBN 979-8-89176-346-3. doi: 10.18653/v1/2025.blackboxnlp-1.14. URL https://aclanthology.org/2025.blackboxnlp-1.14/.

Heimersheim, S. and Nanda, N. How to use and interpret activation patching, 2024. URL https://arxiv.org/abs/2404.15255.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), January 2025. ISSN 1046-8188. doi: 10.1145/3703155. URL https://doi.org/10.1145/3703155.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023. ISSN 0360-0300. doi: 10.1145/3571730. URL https://doi.org/10.1145/3571730.

Kobayashi, G., Kuribayashi, T., Yokoi, S., and Inui, K. Incorporating residual and normalization layers into analysis of masked language models. In Moens, M., Huang, X., Specia, L., and Yih, S. W. (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pp. 4547–4568. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.EMNLP-MAIN.373. URL https://doi.org/10.18653/v1/2021.emnlp-main.373.

Kramár, J., Lieberum, T., Shah, R., and Nanda, N. Atp*: An efficient and scalable method for localizing LLM behaviour to components. *CoRR*, abs/2403.00745, 2024. doi: 10.48550/ARXIV.2403.00745. URL https://doi.org/10.48550/arXiv.2403.00745.

Li, J., Cheng, X., Zhao, X., Nie, J.-Y., and Wen, J.-R. HaluEval: A large-scale hallucination evaluation benchmark for large language models. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 6449–6464, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.397. URL https://aclanthology.org/2023.emnlp-main.397/.

Li, Z.-Z., Zhang, D., Zhang, M.-L., Zhang, J., Liu, Z., Yao, Y., Xu, H., Zheng, J., Wang, P.-J., Chen, X., Zhang, Y., Yin, F., Dong, J., Li, Z., Bi, B.-L., Mei, L.-R., Fang, J., Liang, X., Guo, Z., Song, L., and Liu, C.-L. From system 1 to system 2: A survey of reasoning large language models, 2025. URL https://arxiv.org/abs/2502.17419.

Lin, S., Hilton, J., and Evans, O. TruthfulQA: Measuring how models mimic human falsehoods. In Muresan, S., Nakov, P., and Villavicencio, A. (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3214–3252, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.229. URL https://aclanthology.org/2022.acl-long.229/.

Lindsey, J., Gurnee, W., Ameisen, E., Chen, B., Pearce, A., Turner, N. L., Citro, C., Abrahams, D., Carter, S., Hosmer, B., Marcus, J., Sklar, M., Templeton, A., Bricken, T., McDougall, C., Cunningham, H., Henighan, T., Jermyn, A., Jones, A., Persic, A., Qi, Z., Thompson, T. B., Zimmerman, S., Rivoire, K., Conerly, T., Olah, C., and Batson, J. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

Liu, F., Kandpal, N., and Raffel, C. Attribot: A bag of tricks for efficiently approximating leave-one-out context attribution. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=9kJperA2a4.

Lundberg, S. M. and Lee, S. A unified approach to interpreting model predictions. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 4765–4774, 2017.

Luo, H. and Specia, L. From understanding to utilization: A survey on explainability for large language models, 2024. URL https://arxiv.org/abs/2401.12874.

Ma, M., Zhang, J., Yang, F., Kang, Y., Lin, Q., Yang, T., Rajmohan, S., and Zhang, D. Dover: Intervention-driven auto debugging for llm multi-agent systems, 2025. URL https://arxiv.org/abs/2512.06749.

Maynez, J., Narayan, S., Bohnet, B., and McDonald, R. On faithfulness and factuality in abstractive summarization. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1906–1919, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.173. URL https://aclanthology.org/2020.acl-main.173/.

Meng, K., Bau, D., Andonian, A., and Belinkov, Y. Locating and editing factual associations in gpt. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 17359–17372. Curran Associates, Inc., 2022.

Meng, K., Sharma, A. S., Andonian, A. J., Belinkov, Y., and Bau, D. Mass-editing memory in a transformer. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=MkbcAHIYgyS.

Nanda, N. Attribution patching: Activation patching at industrial scale. 2023. URL https://www.neelnanda.io/mechanistic-interpretability/attribution-patching.

Nanda, N. and Bloom, J. Transformerlens. https://github.com/TransformerLensOrg/TransformerLens, 2022.

Olmo, T., :, Ettinger, A., Bertsch, A., Kuehl, B., Graham, D., Heineman, D., Groeneveld, D., Brahman, F., Timbers, F., Ivison, H., Morrison, J., Poznanski, J., Lo, K., Soldaini, L., Jordan, M., Chen, M., Noukhovitch, M., Lambert, N., Walsh, P., Dasigi, P., Berry, R., Malik, S., Shah, S., Geng, S., Arora, S., Gupta, S., Anderson, T., Xiao, T., Murray, T., Romero, T., Graf, V., Asai, A., Bhagia, A., Wettig, A., Liu, A., Rangapur, A., Anastasiades, C., Huang, C., Schwenk, D., Trivedi, H., Magnusson, I., Lochner, J., Liu, J., Miranda, L. J. V., Sap, M., Morgan, M., Schmitz, M., Guerquin, M., Wilson, M., Huff, R., Bras, R. L., Xin, R., Shao, R., Skjonsberg, S., Shen, S. Z., Li, S. S., Wilde, T., Pyatkin, V., Merrill, W., Chang, Y., Gu, Y., Zeng, Z., Sabharwal, A., Zettlemoyer, L., Koh, P. W., Farhadi, A., Smith, N. A., and Hajishirzi, H. Olmo 3, 2025. URL https://arxiv.org/abs/2512.13961.

OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025a. URL https://arxiv.org/abs/2508.10925.

OpenAI. Introducing gpt-5, 2025b. URL https://openai.com/index/introducing-gpt-5/.

Rosser, J., García, J. L. R., Penha, G., Palla, K., and Bouchard, H. Stream: Scaling up mechanistic interpretability to long context in llms via sparse attention, 2025. URL https://arxiv.org/abs/2510.19875.

Simonyan, K., Vedaldi, A., and Zisserman, A. Deep inside convolutional networks: Visualising image classification models and saliency maps. In Bengio, Y. and LeCun, Y. (eds.), *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*, 2014. URL http://arxiv.org/abs/1312.6034.

Smilkov, D., Thorat, N., Kim, B., Viégas, F. B., and Wattenberg, M. Smoothgrad: removing noise by adding noise. *CoRR*, abs/1706.03825, 2017. URL http://arxiv.org/abs/1706.03825.

Song, P., Han, P., and Goodman, N. A survey on large language model reasoning failures. In *2nd AI for Math Workshop @ ICML 2025*, 2025. URL https://openreview.net/forum?id=hsgMn4KBFG.

Sundararajan, M., Taly, A., and Yan, Q. Axiomatic attribution for deep networks. In Precup, D. and Teh, Y. W. (eds.), *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, volume 70 of *Proceedings of Machine Learning Research*, pp. 3319–3328. PMLR, 2017. URL http://proceedings.mlr.press/v70/sundararajan17a.html.

Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., et al. Kimi k2: Open agentic intelligence, 2025. URL https://arxiv.org/abs/2507.20534.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

Wang, K. R., Variengien, A., Conmy, A., Shlegeris, B., and Steinhardt, J. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL https://openreview.net/forum?id=NpsVSN6o4ul.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning*

Representations, 2024. URL https://openreview.net/forum?id=NG7sS51zVF.

Xu, Z., Xie, S., Lv, Q., Xiao, S., Song, L., Wenjuan, S., and Lin, F. Diagnosing failures in large language models' answers: Integrating error attribution into evaluation framework. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 21148–21165, 2025.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Yao, Y., Zhang, N., Xi, Z., Wang, M., Xu, Z., Deng, S., and Chen, H. Knowledge circuits in pretrained transformers. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 118571–118602. Curran Associates, Inc., 2024. doi: 10.52202/079017-3765.

Yin, K. and Neubig, G. Interpreting language models with contrastive explanations. In Goldberg, Y., Kozareva, Z., and Zhang, Y. (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pp. 184–198. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.EMNLP-MAIN.14. URL https://doi.org/10.18653/v1/2022.emnlp-main.14.

Zhang, F. and Nanda, N. Towards best practices of activation patching in language models: Metrics and methods. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=Hf17y6u9BC.

Zhang, J., Lin, Q., Rajmohan, S., and Zhang, D. From reasoning to answer: Empirical, attention-based and mechanistic insights into distilled DeepSeek r1 models. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 3985–4002, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.198. URL https://aclanthology.org/2025.emnlp-main.198/.

Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., and Wu, Q. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Forty-second International Conference on Machine Learning*, 2025b. URL https://openreview.net/forum?id=GazlTYxZss.

Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., Wang, L., Luu, A. T., Bi, W., Shi, F., and Shi, S. Siren's song in the ai ocean: A survey on hallucination in large language models. *Computational Linguistics*, 51(4):1373–1418, 12 2025c. ISSN 0891-2017. doi: 10.1162/COLI.a.16. URL https://doi.org/10.1162/COLI.a.16.

Zhao, Z., Koishekenov, Y., Yang, X., Murray, N., and Cancedda, N. Verifying chain-of-thought reasoning via its computational graph, 2025. URL https://arxiv.org/abs/2510.09312.

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. Instruction-following evaluation for large language models, 2023. URL https://arxiv.org/abs/2311.07911.

A. Batch-Packed Multi-Target Backpropagation for Attribution Graph Construction

This appendix presents an efficient procedure for constructing attribution-graph edges under Layer-wise Relevance Propagation (LRP). The method leverages recent results showing that LRP-style relevance propagation in transformer models can be implemented via a *modified gradient×input* formulation, enabling direct use of automatic differentiation frameworks such as PyTorch (Achtibat et al., 2024; Arras et al., 2025). Building on this equivalence, we further exploit the batch dimension to propagate relevance from multiple attribution targets simultaneously, substantially reducing computational cost.

Why gradients represent relevance. LRP aims to decompose a scalar attribution objective, such as a contrastive logit difference $\Delta\ell$ as in this work, into contributions from intermediate activations. For non-linear architectures like transformers, recent work has shown that LRP rules can be implemented by *patching the backward pass* with customized local propagation rules. Crucially, these patched backward passes are equivalent to computing a modified gradient×input quantity, allowing relevance to be obtained using standard automatic differentiation (Achtibat et al., 2024; Arras et al., 2025).

Under this formulation, the relevance of a hidden state $\mathbf{h}_l^{(i)} \in \mathbb{R}^d$ is approximated by

$$\mathbf{R}_l^{(i)} \;\approx\; \mathbf{h}_l^{(i)} \odot \frac{\partial \Delta\ell}{\partial \mathbf{h}_l^{(i)}}, \tag{6}$$

where the gradient indicates how sensitive the attribution objective is to the hidden state. Thus, gradients act as the carriers of relevance, and no explicit relevance propagation rules need to be implemented beyond the modified backward pass.

Problem setup. We seek to construct an *attribution graph* whose nodes correspond to hidden-state relevances and whose edges describe how relevance propagates between layers. Fix a source layer s and a target layer t with $s < t$. Let

$$\mathbf{H}^{(s)} \in \mathbb{R}^{1\times n\times d} \tag{7}$$

denote the source-layer hidden states for a sequence of length n, and let

$$\mathbf{H}^{(t)} = F_{s\to t}\left(\mathbf{H}^{(s)}\right) \in \mathbb{R}^{1\times n\times d} \tag{8}$$

denote the corresponding target-layer hidden states, where $F_{s\to t}$ is the composition of transformer submodules between layers s and t.

A single forward and backward pass from $\Delta\ell$ yields gradients

$$\mathbf{g}_l^{(i)} = \frac{\partial \Delta\ell}{\partial \mathbf{h}_l^{(i)}} \tag{9}$$

at every hidden state. These gradients determine node relevances via gradient×input and are stored for later use. At this stage, we know *which* hidden states are relevant, but not *how* relevance flows between hidden states across layers.

Edge attribution via gradient×input. To construct attribution-graph edges, we propagate relevance from a target-layer hidden state $\mathbf{h}_t^{(j)}$ to all source-layer hidden states $\{\mathbf{h}_s^{(i)}\}_{i=1}^n$. We treat the stored gradient $\mathbf{g}_t^{(j)} = \partial\Delta\ell/\partial\mathbf{h}_t^{(j)}$ as a fixed relevance signal. Because automatic differentiation requires a scalar objective, we define the attribution target as the inner product

$$\langle \mathbf{h}_t^{(j)}, \mathbf{g}_t^{(j)} \rangle, \tag{10}$$

which simply selects and weights the hidden dimensions that are relevant for the final decision. Backpropagating this scalar through $F_{s\to t}$ yields gradients with respect to $\mathbf{H}^{(s)}$.

Relevance propagated from target token j to source token i is then computed using the same gradient×input rule:

$$A_{j,i} \;\approx\; \sum_{k=1}^{d} \mathbf{h}_s^{(i,k)} \frac{\partial \langle \mathbf{h}_t^{(j)}, \mathbf{g}_t^{(j)} \rangle}{\partial \mathbf{h}_s^{(i,k)}}. \tag{11}$$

This quantity defines a directed edge from node (s, i) to node (t, j) in the attribution graph.

Batch-packed multi-target backpropagation. Naïvely, computing $A_{j,i}$ for all $j \in \{1, \ldots, n\}$ would require $O(n)$ separate backward passes, one per target token. To avoid this cost, we exploit the batch dimension to process multiple attribution targets simultaneously, following recent work on attribution graph construction with sparse features in transcoders (Hanna et al., 2025).

Let $\mathcal{J} = \{j_1, \ldots, j_B\}$ be a set of target indices. We construct a batched source tensor

$$\widetilde{\mathbf{H}}^{(s)} \in \mathbb{R}^{B \times n \times d}, \qquad \widetilde{\mathbf{H}}^{(s)}_{b,:,:} = \mathbf{H}^{(s)}_{1,:,:}, \tag{12}$$

so that each batch element contains an identical copy of the source-layer hidden states. Forward propagation yields

$$\widetilde{\mathbf{H}}^{(t)} = F_{s \to t}\left(\widetilde{\mathbf{H}}^{(s)}\right) \in \mathbb{R}^{B \times n \times d}. \tag{13}$$

We then define a batched output-gradient tensor $\widetilde{\mathbf{G}}^{(t)} \in \mathbb{R}^{B \times n \times d}$ such that batch element b injects relevance only at its corresponding target index j_b:

$$\widetilde{\mathbf{G}}^{(t)}_{b,j_b,:} = \mathbf{g}^{(j_b)}_t, \qquad \widetilde{\mathbf{G}}^{(t)}_{b,j,:} = \mathbf{0} \;\text{ for } j \neq j_b. \tag{14}$$

A single automatic differentiation call computes the corresponding gradients with respect to $\widetilde{\mathbf{H}}^{(s)}$, yielding B independent relevance propagations in parallel. Applying gradient$\times$input and writing each batch result into the appropriate row produces B rows of the token-to-token interaction matrix.

Thus, by batching attribution targets, we reduce the cost of constructing attribution-graph edges from $O(n)$ backward passes to $O(\lceil n/B \rceil)$ passes, while remaining fully compatible with LRP-style relevance propagation through standard automatic differentiation.

B. Attribution Graph Pruning and Connected Subgraph Extraction

This appendix describes the pruning strategy used to construct a sparse and interpretable *attribution graph* from the dense token-to-token interaction matrices produced by batch-packed multi-target backpropagation (Appendix A). The objective of pruning is to remove low-relevance edges while preserving the dominant relevance flows between hidden states across layers.

Input representation. For each ordered pair of layers (s, t) with $s < t$, we assume access to:

- a token-to-token interaction matrix

$$\mathbf{A}^{(s \to t)} \in \mathbb{R}^{n \times n}, \tag{15}$$

  where $\mathbf{A}^{(s \to t)}_{j,i}$ denotes the relevance propagated from source-layer hidden state $\mathbf{h}^{(i)}_s$ to target-layer hidden state $\mathbf{h}^{(j)}_t$, as defined by the gradient$\times$input formulation in Appendix A;

- source-layer token relevances

$$R^{(s)}_i \;\equiv\; \sum_{k=1}^{d} \mathbf{R}^{(i,k)}_s; \tag{16}$$

- target-layer token relevances

$$R^{(t)}_j \;\equiv\; \sum_{k=1}^{d} \mathbf{R}^{(j,k)}_t. \tag{17}$$

Each nonzero entry of $\mathbf{A}^{(s \to t)}$ corresponds to a candidate directed edge from node (s, i) to node (t, j) in the attribution graph.

Graph construction. We construct a directed graph $\mathcal{G} = (V, E)$, where each node is indexed by a layer–token pair (l, i). For each layer transition (s, t), we:

1. add all source nodes (s, i) such that $|R_i^{(s)}| > 0$;
2. add all target nodes (t, j) such that $|R_j^{(t)}| > 0$;
3. add directed edges $(s, i) \to (t, j)$ for selected entries $\mathbf{A}_{j,i}^{(s\to t)}$ after pruning, with edge weight

$$w_{(s,i)\to(t,j)} = \mathbf{A}_{j,i}^{(s\to t)}. \tag{18}$$

Pruning objectives. The interaction matrices $\mathbf{A}^{(s\to t)}$ are generally dense, making direct graph construction impractical. We therefore prune edges using magnitude-based criteria applied to $|\mathbf{A}^{(s\to t)}|$. Pruning is performed *independently for each layer pair* to preserve local attribution structure and avoid conflating relevance scales across layers.

Let

$$E_{\text{nz}}^{(s\to t)} = \{(j, i) \mid |\mathbf{A}_{j,i}^{(s\to t)}| > 0\} \tag{19}$$

denote the set of nonzero candidate edges, and let

$$M^{(s\to t)} = \sum_{(j,i)\in E_{\text{nz}}^{(s\to t)}} \left|\mathbf{A}_{j,i}^{(s\to t)}\right| \tag{20}$$

denote the total absolute relevance mass for that layer transition.

Pruning modes. We support two pruning strategies:

- *Global threshold pruning.* In this mode, we retain all edges whose absolute relevance exceeds a fixed threshold $\tau > 0$:

$$E_{\text{keep}}^{(s\to t)} = \left\{(j, i) \;\middle|\; \left|\mathbf{A}_{j,i}^{(s\to t)}\right| > \tau\right\}. \tag{21}$$

  This mode enforces a uniform relevance scale across all layer transitions and is most appropriate when attribution magnitudes are comparable across layers.

- *Per-layer cumulative mass pruning.* In this mode, pruning is performed adaptively per layer transition by preserving a fixed fraction of the total relevance mass. Let

$$\{a_1 \geq a_2 \geq \cdots \geq a_K\}$$

  denote the sorted absolute values of all nonzero entries of $\mathbf{A}^{(s\to t)}$, where $K = |E_{\text{nz}}^{(s\to t)}|$. Given a percentile parameter $p \in (0, 1]$, we select the smallest index k^* such that

$$\sum_{k=1}^{k^*} a_k \;\geq\; p \cdot \sum_{k=1}^{K} a_k. \tag{22}$$

  The resulting dynamic threshold is

$$\tau_{s\to t}^{(p)} = a_{k^*}, \tag{23}$$

  and we retain all edges satisfying

$$\left|\mathbf{A}_{j,i}^{(s\to t)}\right| \;\geq\; \tau_{s\to t}^{(p)}. \tag{24}$$

  This strategy guarantees that a fixed proportion of relevance mass is preserved for each layer transition, even when absolute relevance scales differ substantially across layers.

Connected attribution subgraph extraction. After pruning, the attribution graph may be disconnected. Since our attribution objective is the contrastive logit difference at the final token position, we retain only the subgraph that contributes to this prediction. Specifically, we select the target node corresponding to the hidden state at the last layer and last token position, i.e., $(L-1,\, n)$, and extract the induced subgraph consisting of all nodes that can reach the target node via directed paths. All nodes and edges not connected to this target node are discarded.

Resulting graph properties. The final attribution graph $\mathcal{G}_{\text{conn}}$ is sparse, directed, and weighted. Each node (l, i) is annotated with its layer index, token position, and scalar node relevance $R_i^{(l)}$, while each directed edge encodes signed relevance flow between hidden states across layers. By construction, every node and edge in $\mathcal{G}_{\text{conn}}$ lies on at least one relevance path terminating at the final prediction, enabling faithful path tracing, subgraph analysis, and visualization of dominant attribution circuits.

C. Contrast Token Pair Identification

This appendix consolidates the procedures for (i) identifying the first error token (target token) in a model-generated completion, and (ii) selecting an appropriate contrast token for contrastive attribution.

C.1. Target Token Identification

For each failure case, we identify the first output token that causes the reasoning to deviate from the correct trajectory. This token is referred to as the target token. We adopt a semi-automated annotation procedure to identify the target token. Specifically, we prompt multiple powerful LLMs including GPT-5.2-20251211 (OpenAI, 2025b), GPT-OSS-120B (OpenAI, 2025a), DeepSeek-V3.2 (DeepSeek-AI et al., 2025) and Kimi-K2-Thinking (Team et al., 2025), to independently predict the most likely target token for each failure case. The token receiving the majority of votes is selected as the automated result. Human annotators then verify this prediction and correct it when necessary, balancing efficiency with precision. The prompt template used in the above semi-automated annotation procedure is unified across all LLMs to ensure deterministic and reproducible identification of the target token. The full template is shown below (using GAIA2 as an example).

```
version: v1
system: |-
  - You are an expert in localizing error tokens in model\-generated text completions.
  - You will be provided with a prompt and a model\-generated completion, and your task
      is to localize and output the FIRST token in the completion that is erroneous or
      causes the error.
  - As the prompt and completion are taken from a particular dataset, refer to Section
      "Dataset Description" for a brief overview of the dataset.
  - Refer to Section "On the Input Format" for details on how the prompt and completion
      will be presented to you.
  - Follow the "General Guidance \(Dataset Agnostic\)" section for general instructions
      on how to approach the task of error token localization.
  - Refer to the "Dataset Specific Instructions" section for any additional instructions
      that are specific to this dataset.
  - Finally, adhere to the "On the Output Format" section to ensure your response is
      structured correctly.
  - Note that the dataset specific sections \(i.e., "Dataset Description" and "Dataset
      Specific Instructions"\) can be missing for some datasets. In such cases, simply
      skip those sections and proceed with the rest of the instructions.

  # Dataset Description
    Gaia2 is a benchmark dataset for evaluating AI agents inside simulated, dynamic
        environments provided by the Meta Agents Research Environments \(ARE\).
    It contains time\-evolving scenarios with apps/tools \(e.g., messaging, calendar,
        filesystem, shopping, city, contacts\) and multi\-agent interactions.
    Agents must plan, search, and act via tool/API calls under temporal constraints,
        dynamic events, and noise.
    The dataset evaluates dimensions such as Execution, Search, Adaptability, Time,
        Ambiguity, and multi\-agent collaboration.
    For details, see the dataset card and ARE documentation on Gaia2.
  # On the Input Format
  - The input will be provided in the following JSON format:
```

```
  {
  "prompt": "<The prompt text here>",
  "completion": "<The model\-generated completion text here>",
  "indexed_completion": "<The model\-generated completion text with each token indexed,
      i.e., `token_1[index_1]token_2[index_2]`"
  "ground\_truth": "\<The ground truth answer text here\>" \(This field may be absent in
      some cases\)
  }
- An example of the input format is as follows:
  {
    "prompt": "<|im_start|>user
    What is the capital of France?<|im_end|>
    <|im_start|>assistant
    ",
    "completion": "<think> okay, let me see. The capital of France is Lyon. </think>
    \Lyon.",
    "indexed_completion": "<think>[0] okay,[1] let[2] me[3] see.[4] The[5] capital[6]
        of[7] France[8] is[9] Lyon.[10] </think>[11]

    [12] Lyon[13] .[14]",
    "ground_truth": "Paris, the capital of France."
  }
- The "indexed_completion" field provides a tokenized version of the completion, where
    each token is followed by its index in square brackets. This will help you localize
    the position of tokens in the completion.
- Both the "prompt" and "completion" fields may contain special tokens, e.g.,
    "<|im_start|>", "<|im_end|>", "<think>", "</think>", etc. These tokens are part of
    the model's output format, specifically,
  - "<|im_start|>" and "<|im_end|>" denote the start and end of a message in a
      multi-turn conversation, e.g., "<|im_start|>user
  xxx <|im_end|>" indicates the start of a user message, while "<|im_start|>assistant
  xxx <|im_end|>" indicates the start of an assistant message.
  - "<think>" and "</think>" denote the start and end of the model's internal reasoning
      process.
- More on the internal thought process of the model:
- The internal thought process of the model is represented by the reasoning tokens
    between the "<think>" and "</think>" tokens.
- This part reflects the model's reasoning steps before arriving at the final answer.
- The final answer to the prompt is the text that comes AFTER the "</think>" token.
# General Guidance (Dataset Agnostic)
- Refer to the following steps for localizing an erroneous token in the model-generated
    completion:
  - Step 1: Carefully read dataset description and the provided prompt to understand the
      context and requirements.
  - Step 2: If "completion" and "indexed_completion" fields contains reasoning tokens
      (i.e., tokens between "<think>" and "</think>"), seperate the reasoning part from
      the final answer part.
  - Step 3: Examine the final answer part of the completion first to identify any
      errors. If no errors are found in this part, skip the rest of the steps and output
      "<CORRECT>" as the final answer.
  - Step 4: No matther whether the final answer part contains errors, proceed to examine
      the reasoning part for potential errors.
  - Step 5: Once an erroneous token is identified, use the "indexed_completion" field to
      find its index.
- Addiational strict requirements for localizing the erroneous token:
  - Always report the FIRST erroneous token in the "completion" or "indexed_completion"
      field.
  - The localized erroneous token MUST NOT be the special tokens used for formatting
      (e.g., "<|im_start|>", "<|im_end|>", "<think>", "</think>").
  - NEVER treat a token is erroneous just because it is a subword or punctuation mark,
      as the indexed token str does NOT have to be a single word; it can be a subword or
      punctuation mark as per the tokenization used by the model.
- On the difference between examining the final answer part and the reasoning part:
  - For the final answer part (i.e., the part AFTER "</think>"):
    - ALWAYS examine whether the tokens in this part fulfill the format/style
```

```
        requirements specified in the prompt. If you identify a token in this part that
        violates the format/style requirements, you SHOULD consider it as a potential
        erroneous token.
    - ALSO examine the factual correctness and logical consistency of the tokens in this
        part. If you identify a token in this part that is factually incorrect or
        logically inconsistent with the prompt, you SHOULD consider it as a potential
        erroneous token.
  - For the reasoning part (i.e., the part BETWEEN "<think>" and "</think>"):
    - NEVER examine whether the reasoning tokens have fulfilled the format/style
        requirements specified in the prompt, as the reasoning tokens between "<think>"
        and "</think>" are often in a free-form text format. For example, if the prompt
        requires the model to "respond in a poem format", you MUST NOT consider a token
        in the reasoning part as erroneous just because the reasoning tokens are not in a
        poem format.
    - Focus on the factual correctness and logical consistency of the reasoning tokens.
        If you identify a token in this part that is factually incorrect or logically
        inconsistent with the prompt, you SHOULD consider it as a potential erroneous
        token.

# Dataset Specific Instructions
- This dataset uses the ARE simulation where agent messages may trigger tool/API calls
    and receive environment feedback.
- Messages labeled as tool outputs (e.g., segments marked as `role=tool-response` with
    content being tool results) represent environment feedback and MUST NOT be selected
    as erroneous tokens. Only tokens authored by the assistant are eligible.
- Your goal is to localize the FIRST token in the assistant completion that commits the
    agent to a course of action that makes the user request unachievable in the scenario
    context.
- "Unachievable" includes, but is not limited to:
- Invoking a wrong or unavailable tool/app (e.g., writing to filesystem via
    `create__fs` when exploration to locate a folder is required first).
- Assuming nonexistent resources or paths (e.g., hallucinated file/folder IDs or
    directories).
- Attempting UI interactions that the environment does not support (e.g., instructing
    to "open a user interface" when no such UI exists or is permitted).
- Skipping required discovery steps (e.g., failing to locate a folder before listing
    its contents) and immediately issuing a write/irreversible action.
- When such an action spans multiple indexed tokens, localize the very first token that
    begins the erroneous action or argument (e.g., the first token that starts the wrong
    tool name or the hallucinated path string).
- Tie-break rules:
- If both planning text and an ensuing call appear, choose the first token that commits
    execution (e.g., the token that starts the tool name or the imperative "call"), not
    earlier speculative text.
- If multiple independent errors appear, always select the earliest by index.
- IMPORTANT: Ignore input format issues for tools (e.g., minor
    JSON/schema/quoting/whitespace mistakes) unless they change semantics. Such
    surface-form errors should not be selected as the FIRST error if the underlying
    chosen action is already incorrect.
- Likewise, ignore any content in `role=tool-response` segments; they are environment
    outputs, not assistant errors.
- If no erroneous behavior is found and the answer satisfies the prompt and scenario
    constraints, output "<CORRECT>".
# On the Output Format
- The output MUST be in the following JSON format:
  {
  "error_token": "<The localized erroneous token>",
  "token_index": <The index of the erroneous token in the completion>,
  "explanation": "<A brief explanation of why this token is considered erroneous. Less
      than 50 words.>"
  }
- An example of the output format is as follows:
  {
  "error_token": "Lyon",
  "token_index": 10,
```

```
  "explanation": "The token 'Lyon' is erroneous because the correct capital of France is
      Paris, not Lyon. This indicates a factual error in the model's completion."
  }
- Ensure that the "token_index" corresponds to the index provided in the
    "indexed_completion" field.
- The "explanation" should be concise yet informative, providing enough context to
    justify the localization of the error token.
```

C.2. Contrast Token Identification

To analyze the model's preference mechanism, we pair the target token with a contrast token, a plausible alternative that would have led to a more correct continuation. The selection follows the following steps:

1. Assuming the target token is the model's top-1 output token at the failure position (i.e., the token with the highest logit).
2. Examining the model's top-10 predicted tokens at that position. If any of these tokens, when substituted, would plausibly lead to a more correct continuation, we select the highest-ranked such token as the contrast token.
3. If no suitable contrast token is found in the top-10 predictions, we query a stronger sibling model (e.g., Qwen3-1.7B for Qwen3-0.6B, or Qwen3-8B for Qwen3-4B) and inspect its top-10 predictions at the same position. If a valid contrast token is found, we again select the highest-ranked one.
4. If neither model yields a suitable candidate, we manually infer a semantically appropriate contrast token. In cases where multiple plausible candidates exist, we choose the one with the highest rank in the original model's output logits.

D. Sample Expanded Attribution Graphs

For clarity of exposition, the attribution graph shown in the main text is a reduced representation that includes only the dominant layers, nodes, and relevance pathways. Minor layers and low-magnitude edges are omitted to avoid visual clutter. The Figure 8 provides an expanded version of the graph that preserves a larger portion of the model's internal relevance propagation. While the full graph still prunes extremely low-contribution edges for readability, it offers a more exhaustive view of the attribution structure used in our analysis.

In addition to the representative cases discussed above, we also observe another class of attribution graph patterns at MATH dataset, as shown in Figure 9. It further illuminates the model's internal decision dynamics. Specifically, the full attribution graph for this example reveals that the model's preference for the target token is driven overwhelmingly by internal biases, most notably the BOS token, while receiving only minimal contribution from the reasoning chain present in the prompt. The relevance mass rapidly concentrates along a small set of bias-dominated pathways, bypassing the majority of intermediate reasoning tokens. This structure indicates that the model's erroneous decision does not arise from misinterpreting the logical steps in the prompt, but rather from an inherent predisposition encoded in deep layers and positional biases, which suppress the semantic evidence that should have been propagated along the reasoning chain.

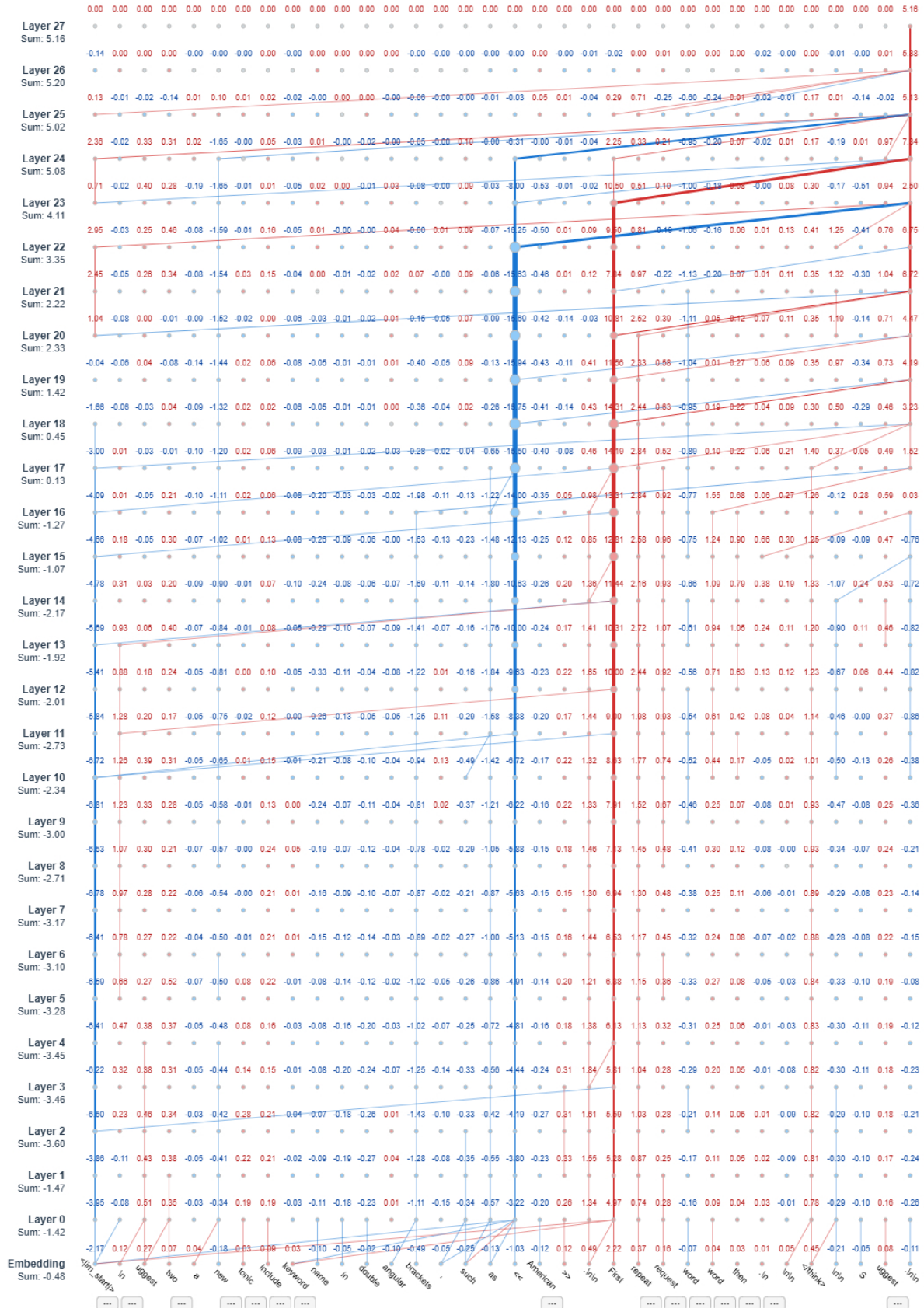

Figure 8. Expanded attribution graph for case in Figure 3.

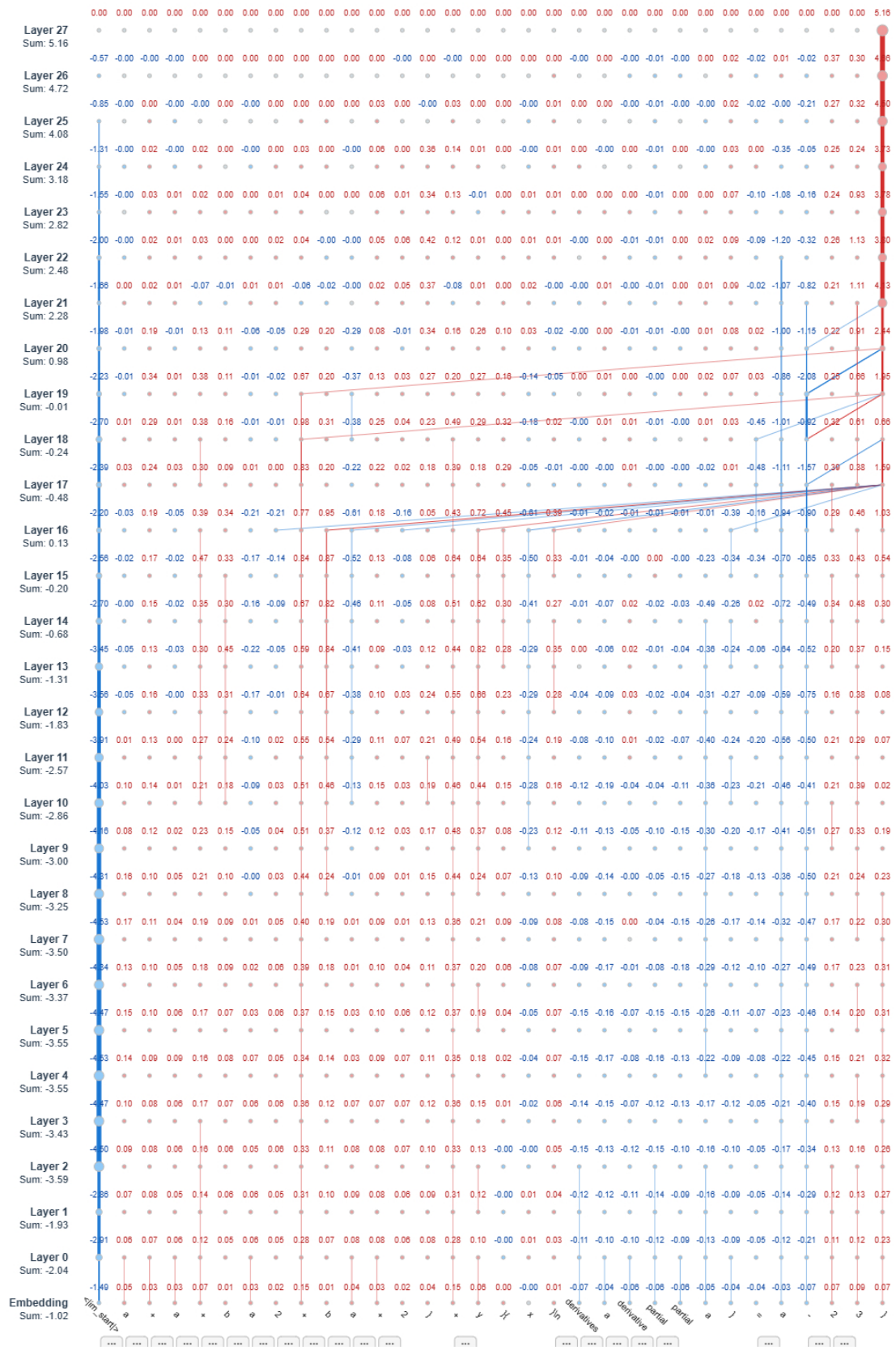

Figure 9. Expanded attribution graph for an example case from MATH, with biases embedded in internal representations.